Is there any difference between the combinations *rg* and *gr*? Try out these commands and you will get the answers. *tr///* is a similar operator in Perl, which is used for transliterating one set of characters to another set. This operator doesn't use regular expressions and, hence, we are not going to discuss the *tr///* operator.

Now it is time for us to develop a few practical regular expressions. Assume you have a text file full of integers and real numbers. Your task is to identify all the integers. Reading data from a text file and identifying its format is very difficult in languages like C where the use of regular expressions is still not widely supported. But you can do it with a single line of code in Perl. To test our Perl one-liner, let us consider a text file *file4.txt* with the following data.

```
-111
2.23
+222
.333
-4.4
0000
555
0001
+6.6
888
77
```

Now, execute the following command to analyse the text file:

```
perl -ne 'if(/(^[+-]?\d+$)/) {print "$1 : Integer\n"}' file4.
txt.
```

All the integers are correctly identified by this regular expression. But a side effect of the definition of this regular expression is that even numbers like 0000, 0001, etc, will be identified as integers. The regex *^[+-]?* uses the metacharacter caret ^ to force a match at the beginning and the metacharacter *?* makes the occurrence of a regular expression optional. So the occurrence of a + or - symbol at the beginning is optional. The metasymbol *\d*, which denotes a decimal digit, is used along with the metacharacter + to denote one or more repetitions of decimal digits as *\d+*. The dollar symbol *$* makes sure that the string should end with a decimal digit. Now let us execute the following Perl one-liner on the same text file:

```
perl -ne 'if(/(^[+-]?\d+\.\d+$)/) {print "$1 : Real
Number\n"}' file4.txt.
```

This regular expression identifies all the real numbers. The trick is identifying the decimal point at the middle with the regular expression **. The regular expression also makes sure that at least one decimal digit occurs before and after the decimal point. Due to this reason, a real number like *.333* will not be detected by this regular expression. Figure 6 shows the output of these regular expressions.

```
# perl -ne 'if(/(^[+-]?\d+$)/) {print "$1 : Integer\n"}' file4.txt
-111 : Integer
+222 : Integer
0000 : Integer
555 : Integer
0001 : Integer
888 : Integer
77 : Integer
# perl -ne 'if(/(^[+-]?\d+\.\d+$)/) {print "$1 : Real Number\n"}' file4.txt
2.23 : Real Number
-4.4 : Real Number
+6.6 : Real Number
```

Figure 6: Regular expressions for numbers

Now it is time for us to encounter the ever-confusing curly bracket metacharacter in regular expressions once again. Curly brackets are used as quantifiers in regular expressions. Consider the regular expression *a{3}* which requires exactly three *a's* for a match. Now execute the Perl one-liner:

```
perl -e '$str = "aaa"; $str  =~ /a{3}/ && print "$str\n"'
```

…and the string *aaa* is printed as expected. What about the command:

```
perl -e '$str = "aaaaaa"; $str  =~ /a{3}/ && print "$str\n"'
```

Again, as expected, we are surprised to see the string *aaaaaa* printed on the screen. In the previous article in this series I have explained the theoretical reasons for this pitfall. I hope the explanation made it clear that regular expressions in theory do not have the ability to count; instead, they just match patterns. By using regular expressions alone it is not possible to count accurately; but, there are certain techniques that can be used to perform certain counting tasks. For example, the regular expression *\d{3}* will identify all strings containing three or more decimal digits, like 333, 4444, 55555, .333, +222, etc, whereas the regular expression *^\d{3}$* will match only those strings with exactly three decimal digits, like 555, 888, etc.

So with that, we reach the end of our discussion. I believe a whole book is necessary to discuss the intricacies of the Perl programming language, and several books are necessary to cover Perl regular expressions. I truly believe that this is not an overstatement. In fact, you need to practice a lot to become masters of Perl regular expressions but you definitely will be rewarded for your hard work. You will uncover knowledge from pieces of text you thought were absolute junk, using tiny regular expressions and Perl. In the next part in this series we will discuss the regular expression syntax of yet another programming language—maybe a language whose regular expression syntax is slightly different from Python and Perl. END

By: Deepu Benson

The author has nearly 16 years of programming experience. He is a free software enthusiast and his area of interest is theoretical computer science. The author maintains a technical blog at *www.computingforbeginners.blogspot.in* and can be reached at *deepumb@hotmail.com*.